!pip install missingno

In [1]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import plotly.express as px
import seaborn as sns
import plotly.figure_factory as ff
import missingno as msno
from sklearn.preprocessing import LabelEncoder
```

In [2]:
```
train_path = "/Users/bhaktipaithankar/Downloads/archive/train_data.csv"
train_df = pd.read_csv(train_path)
test_path = "/Users/bhaktipaithankar/Downloads/archive/test_data.csv"
test_df = pd.read_csv(test_path)
```

In [3]:
```
train_df.head(50)
```

								education	marriage	apart
45	5045654	F	Y	Y	0	157500.0	Working	Secondary / secondary special	Married	Ho apart
46	5112850	F	Y	Y	0	180000.0	Working	Secondary / secondary special	Married	Ho apart
47	5058365	F	N	Y	0	108000.0	Pensioner	Secondary / secondary special	Separated	Ho apart
48	5058521	F	N	Y	0	405000.0	Commercial associate	Secondary / secondary special	Married	pa
49	5062211	F	N	Y	0	112500.0	Commercial associate	Higher education	Married	Ho apart

In [4]:
```
test_df.head(50)
```

Out[4]:

	ID	Gender	Has a car	Has a property	Children count	Income	Employment status	Education level	Marital status	Dw
0	5091261	F	N	Y	0	202500.0	State servant	Secondary / secondary special	Separated	Ho apar
1	5096963	M	Y	N	0	675000.0	Commercial associate	Higher education	Married	Ho apar
2	5087880	F	N	N	0	234000.0	State servant	Higher education	Civil marriage	Ho apar
3	5021949	F	Y	Y	0	445500.0	Commercial associate	Higher education	Married	Ho apar
4	5105705	F	Y	N	0	225000.0	Working	Secondary / secondary special	Married	Mur apar

In [5]:
```
train_df.tail(50)
```

Out[5]:

	ID	Gender	Has a car	Has a property	Children count	Income	Employment status	Education level	Marital status	
29115	5053875	M	N	Y	0	112500.0	Commercial associate	Secondary / secondary special	Civil marriage	a
29116	5048289	M	Y	Y	0	180000.0	Working	Higher education	Married	a
29117	5010102	F	Y	Y	2	306000.0	State servant	Higher education	Married	a
29118	5087480	M	Y	Y	0	157500.0	Working	Secondary / secondary special	Married	a
29119	5096417	F	N	N	0	76500.0	Working	Secondary / secondary	Married	a

In [6]:
```
train_df.shape
```

Out[6]: (29165, 20)

In [7]:
```
train_df.duplicated().any()
```

Out[7]: False

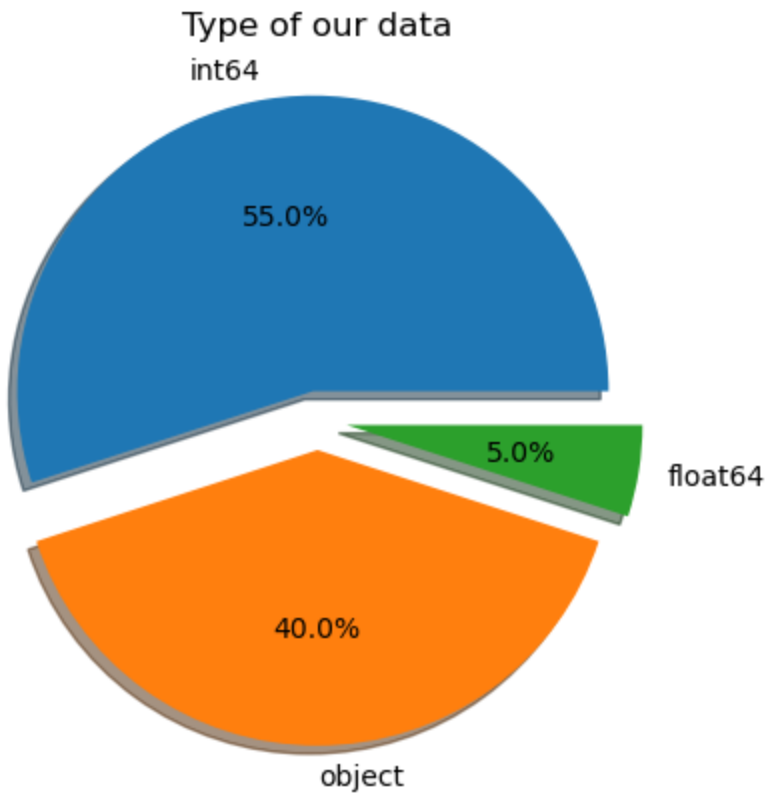
In [8]:
```
train_df.nunique()
```

Out[8]:
```
ID                     29165
Gender                     2
Has a car                  2
Has a property             2
Children count             9
Income                   259
Employment status          5
Education level            5
Marital status             5
Dwelling                   6
Age                     6794
Employment length       3483
Has a mobile phone         1
Has a work phone           2
Has a phone                2
Has an email               2
Job title                 18
Family member count       10
Account age               61
Is high risk               2
dtype: int64
```

In [9]:
```
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29165 entries, 0 to 29164
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   29165 non-null  int64
 1   Gender               29165 non-null  object
 2   Has a car            29165 non-null  object
 3   Has a property       29165 non-null  object
 4   Children count       29165 non-null  int64
 5   Income               29165 non-null  float64
 6   Employment status    29165 non-null  object
 7   Education level      29165 non-null  object
 8   Marital status       29165 non-null  object
 9   Dwelling             29165 non-null  object
 10  Age                  29165 non-null  int64
 11  Employment length    29165 non-null  int64
 12  Has a mobile phone   29165 non-null  int64
 13  Has a work phone     29165 non-null  int64
 14  Has a phone          29165 non-null  int64
 15  Has an email         29165 non-null  int64
 16  Job title            20138 non-null  object
 17  Family member count  29165 non-null  int64
 18  Account age          29165 non-null  int64
 19  Is high risk         29165 non-null  int64
dtypes: float64(1), int64(11), object(8)
memory usage: 4.5+ MB
```

In [10]:
```
train_df.dtypes.value_counts().plot.pie(explode=[0.1,0.1,0.1],autopct='%
plt.title('Type of our data');
```



In [11]:
```
print("Missing values in train data:\n", train_df.isnull().sum())
```

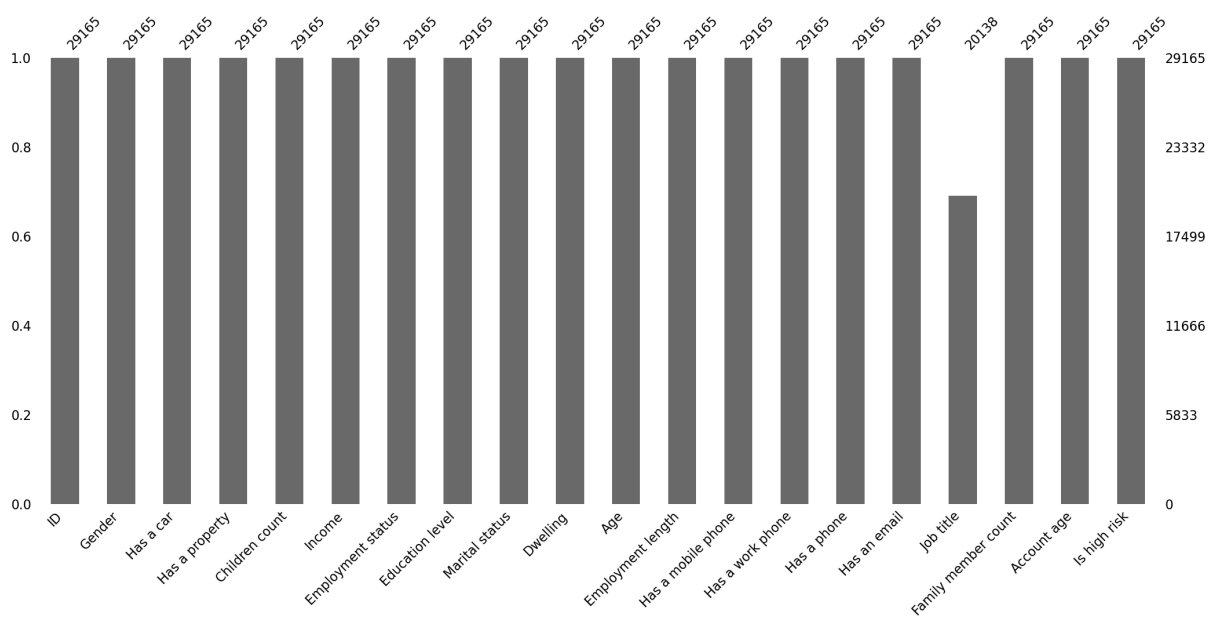
```
Missing values in train data:
 ID                        0
Gender                    0
Has a car                 0
Has a property            0
Children count            0
Income                    0
Employment status         0
Education level           0
Marital status            0
Dwelling                  0
Age                       0
Employment length         0
Has a mobile phone        0
Has a work phone          0
Has a phone               0
Has an email              0
Job title              9027
Family member count       0
Account age               0
Is high risk              0
dtype: int64
```

In [12]:
```
print("Missing values in test data:\n", test_df.isnull().sum())
```

```
Missing values in test data:
 ID                        0
Gender                     0
Has a car                  0
Has a property             0
Children count             0
Income                     0
Employment status          0
Education level            0
Marital status             0
Dwelling                   0
Age                        0
Employment length          0
Has a mobile phone         0
Has a work phone           0
Has a phone                0
Has an email               0
Job title               2296
Family member count        0
Account age                0
Is high risk               0
dtype: int64
```

In [13]:
```
msno.bar(train_df)
```

Out[13]: <Axes: >



In [14]:
```
train_df['Job title'].unique()
```

Out[14]:
```
array(['Core staff', 'Accountants', 'Laborers', 'Managers', nan,
       'Sales staff', 'Medicine staff', 'High skill tech staff',
       'HR staff', 'Low-skill Laborers', 'Drivers', 'Secretaries',
       'Cleaning staff', 'Cooking staff', 'Security staff',
       'Private service staff', 'IT staff', 'Waiters/barmen staff',
       'Realty agents'], dtype=object)
```

In [15]:
```
train_df["Job title"].fillna("unknown",inplace=True)

test_df["Job title"].fillna("unknown",inplace=True)
```

In [16]:
```
print(train_df.columns)
```

```
Index(['ID', 'Gender', 'Has a car', 'Has a property', 'Children count',
       'Income', 'Employment status', 'Education level', 'Marital statu
s',
       'Dwelling', 'Age', 'Employment length', 'Has a mobile phone',
       'Has a work phone', 'Has a phone', 'Has an email', 'Job title',
       'Family member count', 'Account age', 'Is high risk'],
      dtype='object')
```

In [17]:
```
train_df = train_df.drop(columns=["Has a mobile phone"])
```

In [18]:
```
print(train_df.columns.tolist())
```

```
['ID', 'Gender', 'Has a car', 'Has a property', 'Children count', 'Inco
me', 'Employment status', 'Education level', 'Marital status', 'Dwellin
g', 'Age', 'Employment length', 'Has a work phone', 'Has a phone', 'Has
an email', 'Job title', 'Family member count', 'Account age', 'Is high
risk']
```

In [19]:
```
print(test_df.columns)
```

```
Index(['ID', 'Gender', 'Has a car', 'Has a property', 'Children count',
       'Income', 'Employment status', 'Education level', 'Marital statu
s',
       'Dwelling', 'Age', 'Employment length', 'Has a mobile phone',
       'Has a work phone', 'Has a phone', 'Has an email', 'Job title',
       'Family member count', 'Account age', 'Is high risk'],
      dtype='object')
```

In [20]:
```
train_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29165 entries, 0 to 29164
Data columns (total 19 columns):
 #   Column               Non-Null Count  Dtype  
---  ------               --------------  -----  
 0   ID                   29165 non-null  int64  
 1   Gender               29165 non-null  object 
 2   Has a car            29165 non-null  object 
 3   Has a property       29165 non-null  object 
 4   Children count       29165 non-null  int64  
 5   Income               29165 non-null  float64
 6   Employment status    29165 non-null  object 
 7   Education level      29165 non-null  object 
 8   Marital status       29165 non-null  object 
 9   Dwelling             29165 non-null  object 
 10  Age                  29165 non-null  int64  
 11  Employment length    29165 non-null  int64  
 12  Has a work phone     29165 non-null  int64  
 13  Has a phone          29165 non-null  int64  
 14  Has an email         29165 non-null  int64  
 15  Job title            29165 non-null  object 
 16  Family member count  29165 non-null  int64  
 17  Account age          29165 non-null  int64  
 18  Is high risk         29165 non-null  int64  
dtypes: float64(1), int64(10), object(8)
memory usage: 4.2+ MB
```

In [21]:
```
test_df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 7292 entries, 0 to 7291
Data columns (total 20 columns):
 #   Column               Non-Null Count  Dtype
---  ------               --------------  -----
 0   ID                   7292 non-null   int64
 1   Gender               7292 non-null   object
 2   Has a car            7292 non-null   object
 3   Has a property       7292 non-null   object
 4   Children count       7292 non-null   int64
 5   Income               7292 non-null   float64
 6   Employment status    7292 non-null   object
 7   Education level      7292 non-null   object
 8   Marital status       7292 non-null   object
 9   Dwelling             7292 non-null   object
 10  Age                  7292 non-null   int64
 11  Employment length    7292 non-null   int64
 12  Has a mobile phone   7292 non-null   int64
 13  Has a work phone     7292 non-null   int64
 14  Has a phone          7292 non-null   int64
 15  Has an email         7292 non-null   int64
 16  Job title            7292 non-null   object
 17  Family member count  7292 non-null   float64
 18  Account age          7292 non-null   float64
 19  Is high risk         7292 non-null   int64
dtypes: float64(3), int64(9), object(8)
memory usage: 1.1+ MB
```

In [22]:
```
train_df.describe()
```

Out[22]:

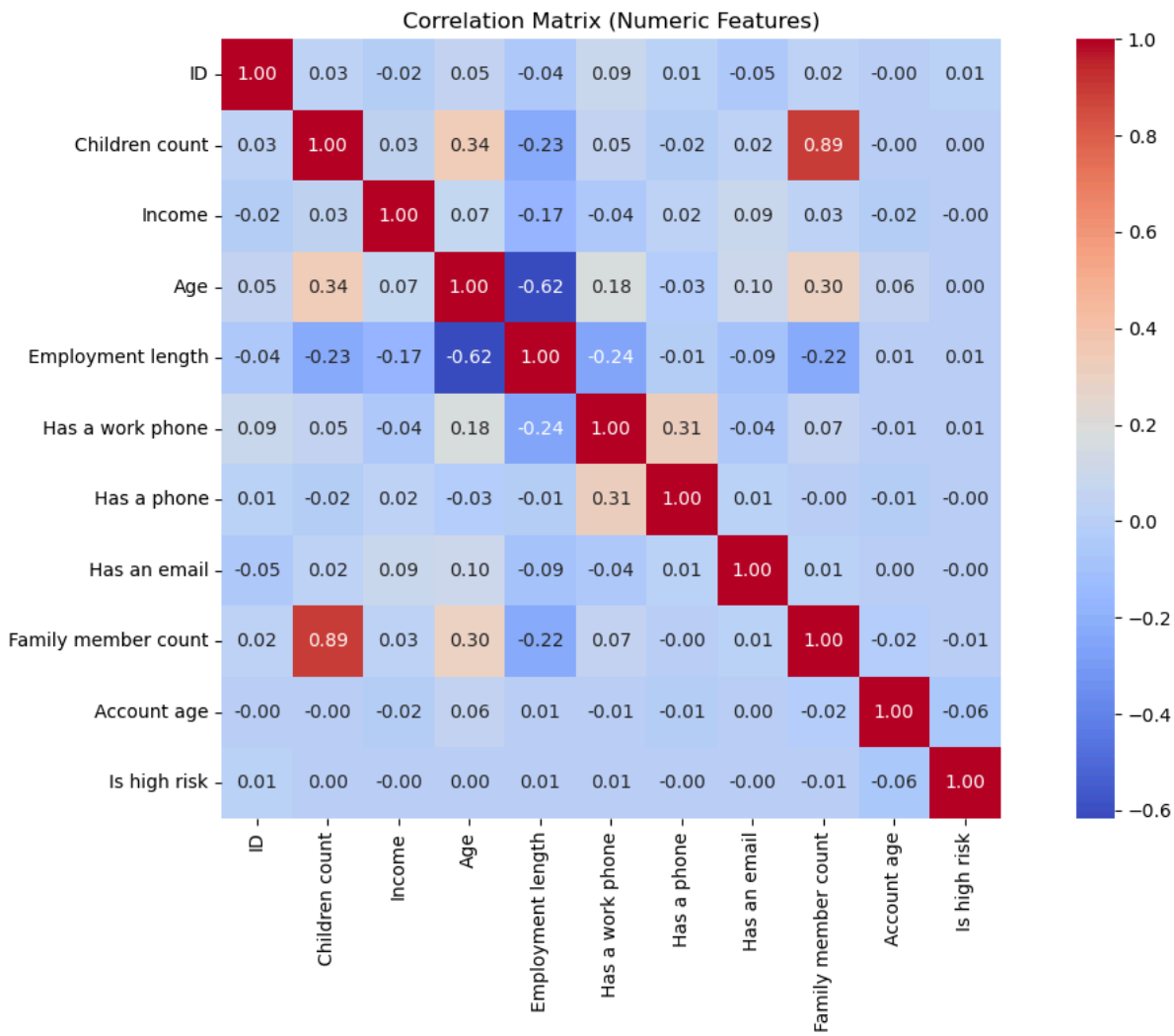
	ID	Children count	Income	Age	Employment length	Has a work phone	H
count	2.916500e+04	29165.000000	2.916500e+04	29165.000000	29165.000000	29165.000000	29
mean	5.078232e+06	0.430790	1.868904e+05	-15979.477490	59257.761255	0.224310	
std	4.182400e+04	0.741882	1.014096e+05	4202.997485	137655.883458	0.417134	
min	5.008804e+06	0.000000	2.700000e+04	-25152.000000	-15713.000000	0.000000	
25%	5.042047e+06	0.000000	1.215000e+05	-19444.000000	-3153.000000	0.000000	
50%	5.074666e+06	0.000000	1.575000e+05	-15565.000000	-1557.000000	0.000000	
75%	5.114629e+06	1.000000	2.250000e+05	-12475.000000	-412.000000	0.000000	
max	5.150485e+06	19.000000	1.575000e+06	-7705.000000	365243.000000	1.000000	

In [23]:
```
# Compute the correlation matrix after dropping the "Has a mobile phone"
correlation_matrix = train_df.corr(numeric_only=True)
correlation_matrix
```

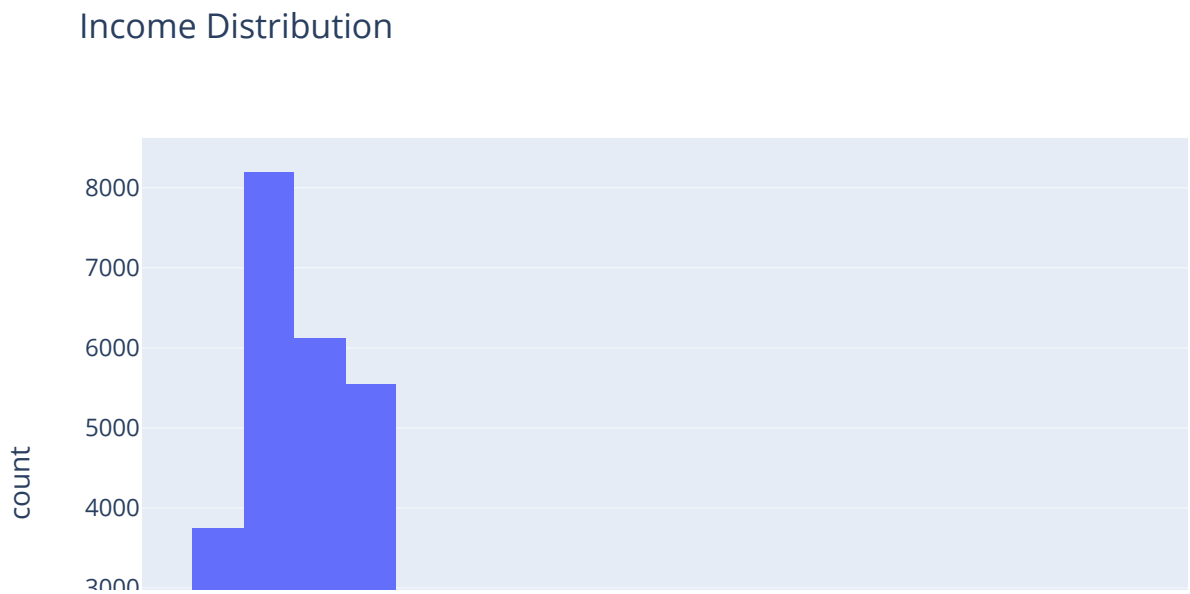
Out[23]:

	ID	Children count	Income	Age	Employment length	Has a work phone	Has a phone	Has er
ID	1.000000	0.026291	-0.016257	0.053120	-0.037380	0.085670	0.013915	-0.048
Children count	0.026291	1.000000	0.034991	0.339080	-0.230468	0.048727	-0.015110	0.015
Income	-0.016257	0.034991	1.000000	0.068156	-0.169940	-0.036601	0.021516	0.091
Age	0.053120	0.339080	0.068156	1.000000	-0.616299	0.177483	-0.027621	0.104
Employment length	-0.037380	-0.230468	-0.169940	-0.616299	1.000000	-0.242055	-0.012511	-0.085
Has a work phone	0.085670	0.048727	-0.036601	0.177483	-0.242055	1.000000	0.312419	-0.035
Has a phone	0.013915	-0.015110	0.021516	-0.027621	-0.012511	0.312419	1.000000	0.011
Has an email	-0.048210	0.015116	0.091500	0.104728	-0.085673	-0.035741	0.011367	1.000
Family member count	0.024724	0.888676	0.025338	0.301941	-0.220102	0.065130	-0.002008	0.013
Account age	-0.001453	-0.003722	-0.016435	0.057467	0.005643	-0.009882	-0.013896	0.002
Is high risk	0.011941	0.000369	-0.001760	0.002422	0.005398	0.005115	-0.000112	-0.004

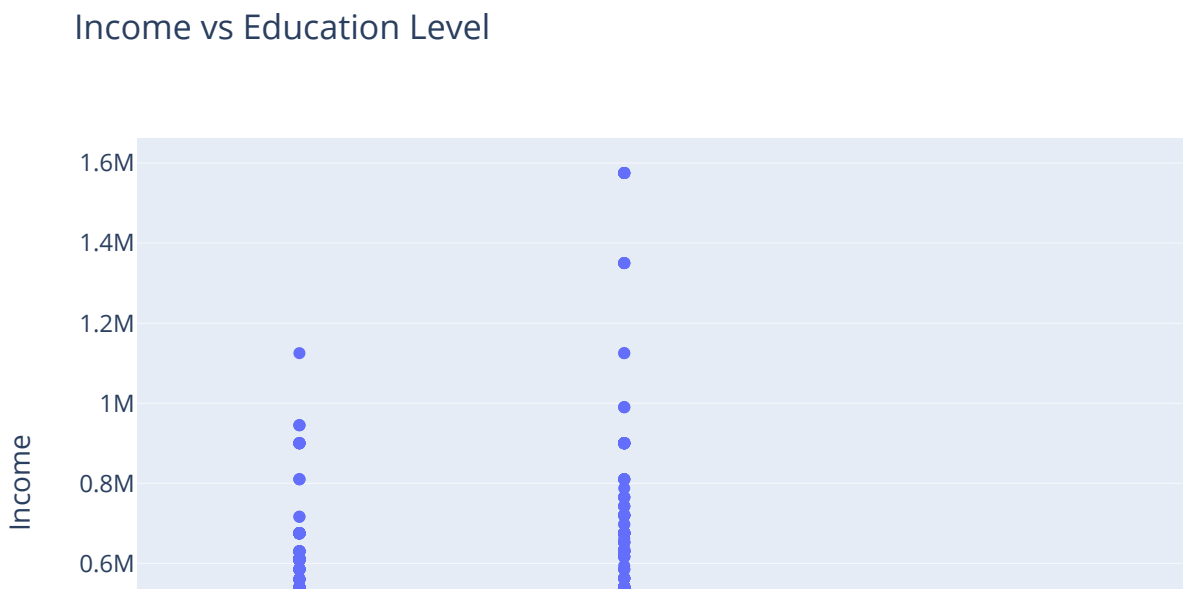
In [24]:
```
# Plotting the correlation matrix
plt.figure(figsize=(12, 8))
sns.heatmap(correlation_matrix, annot=True, fmt=".2f", cmap="coolwarm",
plt.title("Correlation Matrix (Numeric Features)")
plt.tight_layout()
plt.show()
```



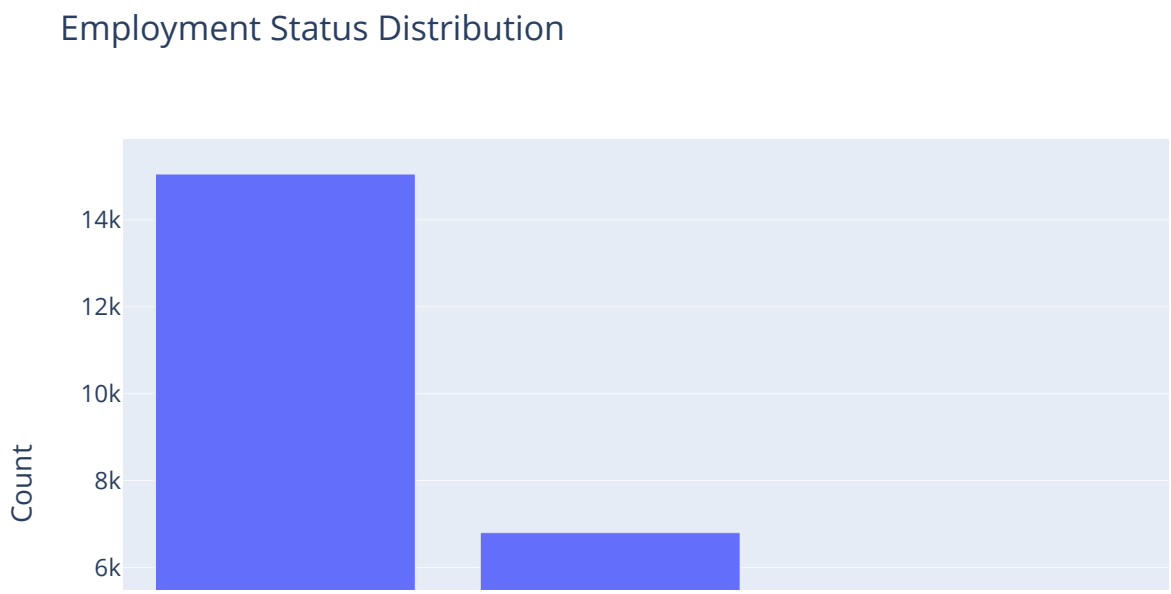
In [25]:
```
# 1. Distribution of Income
fig1 = px.histogram(train_df, x="Income", nbins=50, title="Income Distri
fig1.show()
```



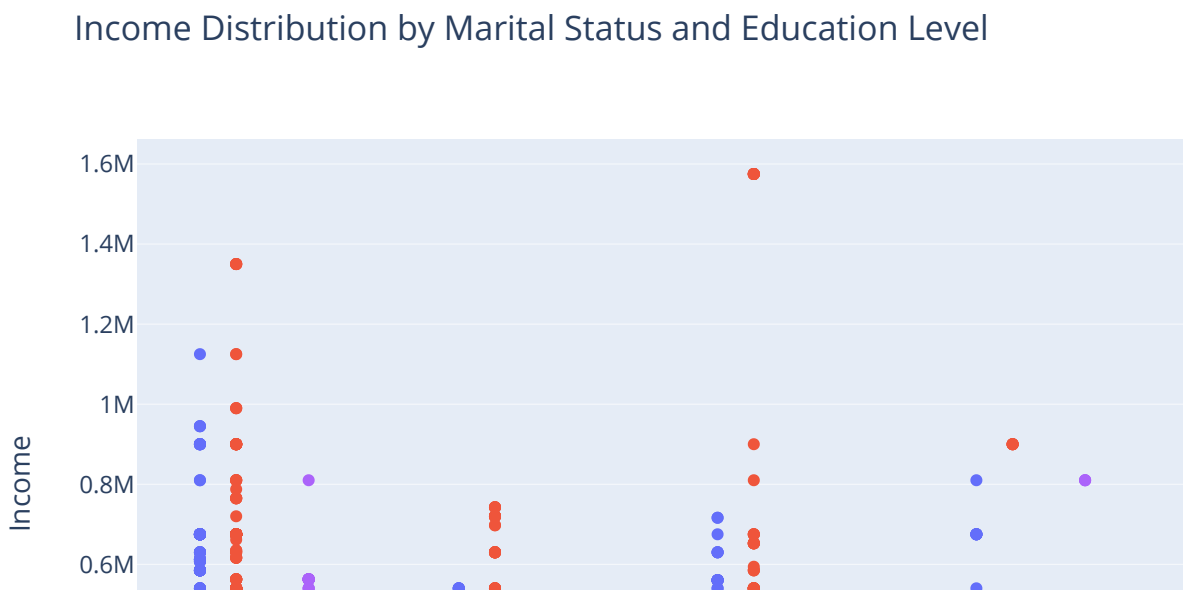
In [26]:
```
# 2. Box plot of Income by Education level
fig2 = px.box(train_df, x="Education level", y="Income", title="Income v
fig2.show()
```



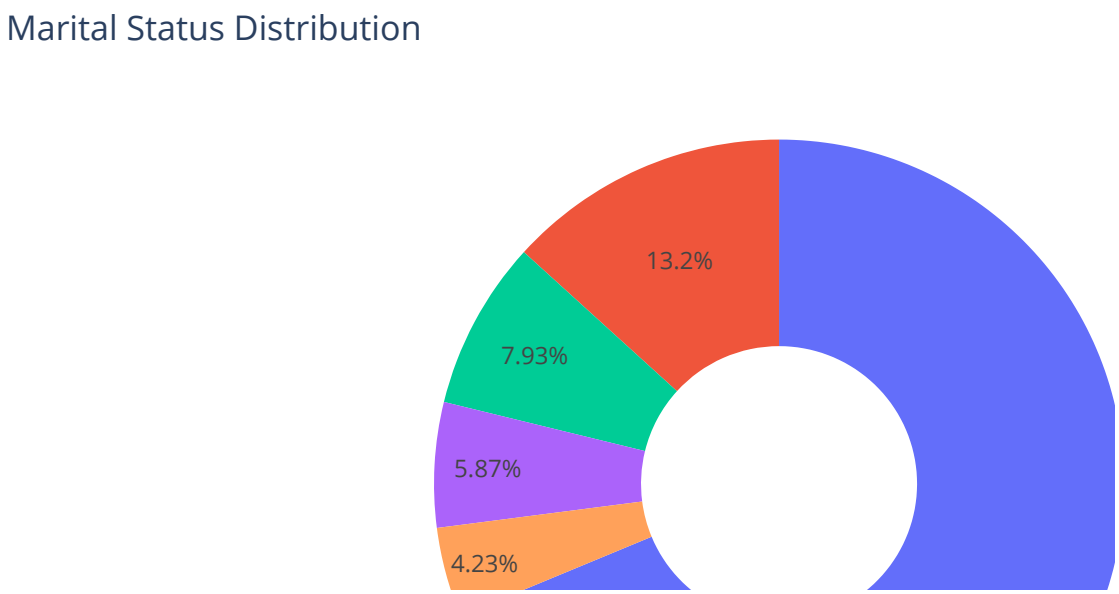
In [27]:
```
# 3. Bar chart: Count of Employment Status
fig3 = px.bar(train_df["Employment status"].value_counts().reset_index()
              x="index", y="Employment status",
              title="Employment Status Distribution",
              labels={"index": "Employment Status", "Employment status":
fig3.show()
```



In [28]:
```
fig = px.box(train_df, x="Marital status", y="Income", color="Education
             hover_data=["Employment status", "Age"],
             title="Income Distribution by Marital Status and Education
fig.show()
```



In [29]:
```
import plotly.express as px
fig = px.pie(train_df, names="Marital status", title="Marital Status Dis
fig.show()
```



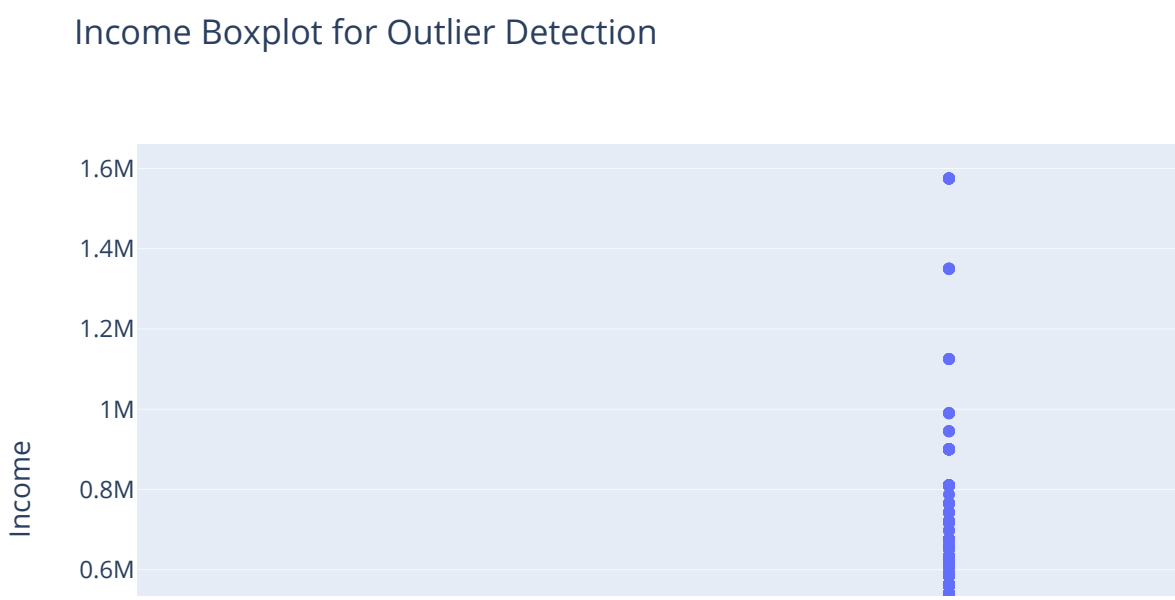
In [38]:
```
from scipy.stats import zscore

# Compute z-scores
z_scores = train_df[["Income", "Age", "Employment length"]].apply(zscore

# Filter rows with any z-score above 3 or below -3 (common threshold)
outliers_z = train_df[(abs(z_scores) > 3).any(axis=1)]

print(f"Z-score outliers found: {outliers_z.shape[0]}")
```

```
Z-score outliers found: 405
```

In [39]:
```
from sklearn.ensemble import IsolationForest

# Use only numerical columns
num_df = train_df.select_dtypes(include='number')

# Fit model
iso = IsolationForest(contamination=0.01, random_state=42)
train_df["Anomaly"] = iso.fit_predict(num_df)

# -1 means anomaly, 1 means normal
anomalies = train_df[train_df["Anomaly"] == -1]
print(f"Anomalies detected by Isolation Forest: {anomalies.shape[0]}")
```

```
Anomalies detected by Isolation Forest: 292
```

In [40]:
```
import plotly.express as px
fig = px.box(train_df, y="Income", title="Income Boxplot for Outlier Det
fig.show()
```



In [41]:
```
# Clean column names
train_df.columns = train_df.columns.str.strip()

# Drop the "Has a mobile phone" column if present
train_df.drop(columns=["Has a mobile phone"], inplace=True, errors='igno

# Drop the "Anomaly" column if present
clean_df = train_df.drop(columns=["Anomaly"], errors='ignore')

# Save the cleaned dataset to Excel
excel_path = "/Users/bhaktipaithankar/Downloads/archive/train_data.xlsx"
clean_df.to_excel(excel_path, index=False)

excel_path
```

Out[41]:
```
'/Users/bhaktipaithankar/Downloads/archive/train_data.xlsx'
```

In [42]:
```
# Re-import necessary libraries after reset
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.impute import SimpleImputer
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, roc_auc_score
import matplotlib.pyplot as plt
import seaborn as sns

# Drop uninformative columns
columns_to_drop = ["ID", "Has a mobile phone", "Job title"]
train_df.drop(columns=columns_to_drop, inplace=True, errors='ignore')

# Separate features and target
X = train_df.drop(columns=["Is high risk"])
y = train_df["Is high risk"]

# Encode categorical features
combined = X.copy()
categorical_cols = combined.select_dtypes(include="object").columns
encoder = LabelEncoder()
for col in categorical_cols:
    combined[col] = encoder.fit_transform(combined[col].astype(str))

# Impute missing values
imputer = SimpleImputer(strategy="median")
X_imputed = imputer.fit_transform(combined)

# Split into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_imputed, y, test_siz

# Define models
models = {
    "Random Forest": RandomForestClassifier(random_state=42),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Support Vector Machine": SVC(probability=True)
}

# Evaluate models
results = []
predictions_table = pd.DataFrame()

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_val)
    y_proba = model.predict_proba(X_val)[:, 1]
    acc = accuracy_score(y_val, y_pred)
    auc = roc_auc_score(y_val, y_proba)

    results.append({"Model": name, "Accuracy": acc, "ROC-AUC": auc})
```

```
    predictions_table[name] = y_pred

# Add actual labels for reference
predictions_table["Actual"] = y_val.reset_index(drop=True)

# Display predictions
predictions_table.head(5)
```

Out[42]:

	Random Forest	Logistic Regression	Decision Tree	K-Nearest Neighbors	Support Vector Machine	Actual
0	0	0	0	0	0	0
1	0	0	0	0	0	0
2	0	0	0	0	0	0
3	0	0	0	0	0	0
4	0	0	0	0	0	0

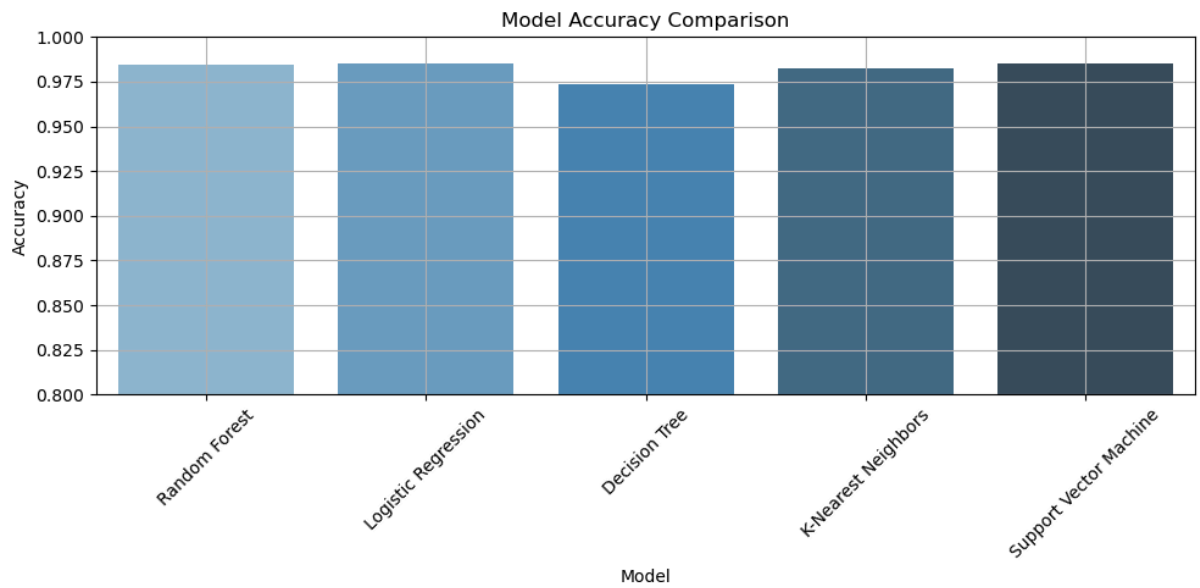
In [43]:

```
results_df = pd.DataFrame(results)

# Display the table of accuracy and ROC-AUC scores
print("Model Performance Metrics:")
display(results_df.sort_values(by="Accuracy", ascending=False))
```

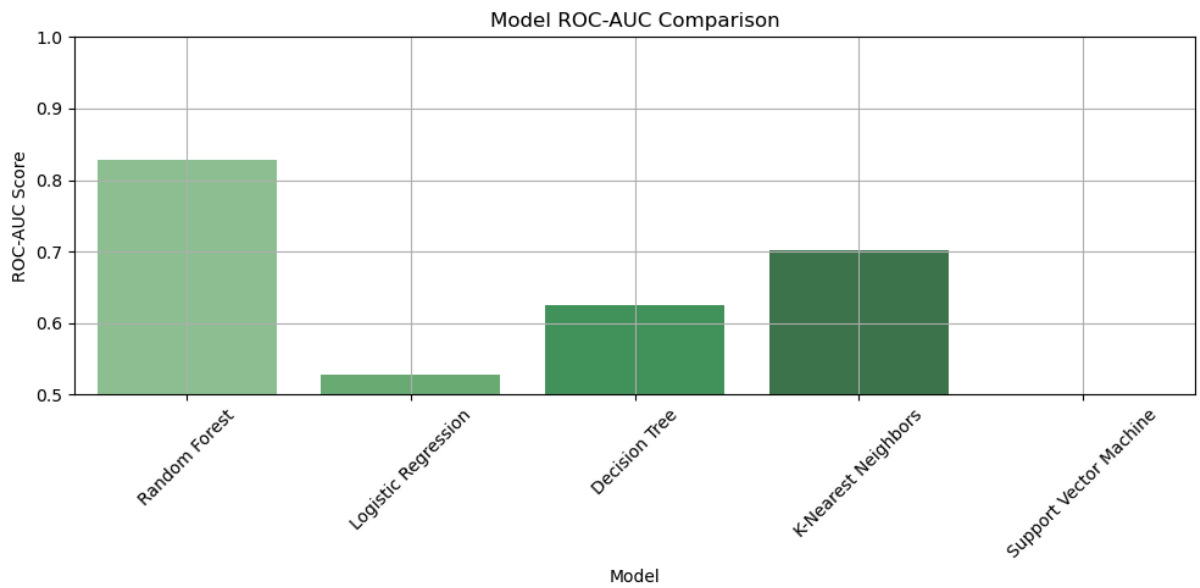
```
Model Performance Metrics:
```

	Model	Accuracy	ROC-AUC
0	Random Forest	0.985256	0.817797
1	Logistic Regression	0.985256	0.527208
4	Support Vector Machine	0.985256	0.566622
3	K-Nearest Neighbors	0.982342	0.702058
2	Decision Tree	0.973598	0.619561

In [36]:
```
# Plotting Accuracy
plt.figure(figsize=(10, 5))
sns.barplot(data=results_df, x="Model", y="Accuracy", palette="Blues_d")
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy")
plt.ylim(0.8, 1.0)
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [37]:
```
plt.figure(figsize=(10, 5))
sns.barplot(data=results_df, x="Model", y="ROC-AUC", palette="Greens_d")
plt.title("Model ROC-AUC Comparison")
plt.ylabel("ROC-AUC Score")
plt.ylim(0.5, 1.0)
plt.xticks(rotation=45)
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [ ]: